

Rules vs. model — the actual measured result

The comparison `docs/MODEL_BATTERY_PLAN.md` §2 describes, run for real on 2026-09-18: `openai/gpt-4o-mini`, live, full 50-case / 118-trial eval set, run twice — once with `DECISION_MODE="rules"`, once with `"model"` — nothing else different. This is the actual evidence behind the team's D0/D6 architecture argument, not a projection.

Raw results committed at `A2_scaffold/results_rules_gpt4o-mini.json` and `A2_scaffold/results_model_gpt4o-mini.json`. Reproduce the table below with `python3 A2_scaffold/compare_modes.py results_rules_gpt4o-mini.json results_model_gpt4o-mini.json`.

The headline numbers

	rules	model
trials	118	118
pass rate (exact code check)	78.0%	37.3%
cost (full run)	US\$0.1084	US\$0.1311
median turns	2.0	2.0
worst-case turns	4	5
<code>route_mismatch</code> (blocked a book)	0	0
<code>route_mismatch_nonblocking</code> (logged only)	12	17

Nothing hit a guardrail in either run (`stopped_by` was `null` for all 236 trials) - every difference here is decision quality and cost, not loop failures or budget/step caps.

Do not stop at the headline number - it is misleading on its own

Broke down every failing trial in BOTH runs by what actually mismatched - not just `model` mode. **Correction (2026-09-18, caught the same day this was first written up):** the first version of this doc compared `model` mode's decision-level accuracy against `rules` mode's RAW pass rate, which happened to match numerically (both ~78%) by coincidence. That is not an apples-to-apples comparison. Computing `rules` mode's own decision-level breakdown properly:

Category	rules	model	What it means
Decision itself was wrong	12	26	A real reasoning miss - the agent concluded the wrong outcome.
Decision right, trigger <i>label</i> wrong	13	47	e.g. <code>'sudden visual loss'</code> instead of the answer key's <code>'red_flag_term'</code> - same finding, different words.

Category	rules	model	What it means
Decision right, booked slot wrong	1	1	Right call to book; wrong clinic/date/time.
Decision-level accuracy	89.8% (106/118)	78.0% (92/118)	Trigger-labelling mismatches set aside.

`rules` mode has **SOME trigger-labelling drift too** (13 trials) - even though `prompt.py`'s `RULES["B_rules"]` tells the model to echo `resolve_routing()`'s trigger string verbatim, `gpt-4o-mini` still occasionally paraphrases it instead of copying it exactly. Worth noting as its own small finding.

The real, correctly-compared gap: 89.8% vs 78.0%, an 11.8-point difference - real, but far smaller than the raw 78.0%-vs-37.3% headline numbers suggest, and NOT "statistically identical" as this doc originally (incorrectly) claimed.

Why the gap is mostly a labelling artifact, not a reasoning gap

"rules" mode hands the model `resolve_routing()`'s already-computed `trigger` string and tells it to report that value faithfully - `prompt.py`'s `RULES["B_rules"]` text is explicit about this. "model" mode gives the model only the raw facts (`red_flag_term`, `right_department`, `missing_tests`, `existing_appointments`) and asks it to derive and *name* the trigger itself, in its own words. The answer key's `code_check()` does an exact string match on `trigger`. A model that correctly identifies "there's a red flag, escalate" but writes `'sudden visual loss'` (the actual red-flag phrase it saw) instead of the category name `'red_flag_term'` gets marked wrong by the code check even though a human reading the record would call it right - which is exactly what the `must_record` judgement-check items are for (see `docs/HOW_IT_WORKS.pdf`'s section on the two kinds of check).

What this actually supports for the D0/D6 argument

- **Rules mode has a real, moderate decision-quality advantage (89.8% vs 78.0%, +11.8 points), not just a labelling-reliability one.** Handing the model a pre-resolved answer to echo genuinely produces fewer wrong DECISIONS, not only cleaner labels - `model` mode's 26 wrong decisions vs `rules` mode's 12 is more than double.
- **Exact-label reliability is a SEPARATE, larger advantage on top of that:** 13 trigger-only misses in `rules` mode vs 47 in `model` mode. Both effects are real and both matter for the argument - don't collapse them into one claim.
- **Cost favours rules mode too** (\$0.1084 vs \$0.1311) - a shorter, more directive prompt plus not needing to re-derive four gates from scratch.
- `route_mismatch_nonblocking` (**17 vs 12**) confirms "model" mode disagrees with the mechanical protocol-only answer more often than "rules" mode does - expected, since "rules" mode is largely just reporting that same mechanical answer back.

Caveats - keep these attached to the number

- **One cheap-tier model only.** This is `gpt-4o-mini`'s specific trigger-labelling behaviour, not a universal law about rules vs. model reasoning. A different model (especially a more capable or instruction-tuned one) may label triggers more consistently with the answer key, closing some or all of this gap.
 - **Not the cross-model battery.** This is the separate, single-model rules-vs-model comparison from `docs/MODEL_BATTERY_PLAN.md` §2 - do not present it as D5(b) cross-model evidence.
 - **Report all four numbers, for both modes.** Raw pass rate (78.0% rules / 37.3% model) and decision-level accuracy (89.8% rules / 78.0% model) are all real and all belong in the write-up - reporting only the raw numbers overstates the gap; reporting only decision-level accuracy understates rules mode's advantage.
-

Written up 2026-09-18. See `docs/CHANGELOG.md` for the JSON-compliance fix and the unknown-tool-name fix that made this run possible - the first two attempts at this exact comparison were unusable data, not real measurements. See `docs/MODEL_BATTERY_PLAN.md` for the frontier-tier (`claude-opus-5`) result, run separately.